APUNTES UNIVERSITARIOS

Foundations of Machine Learning and Deep Learning

Apuntes del Tema 8: Descenso de Gradiente, Métodos Acelerados y Adaptativos

19 de octubre de 2026

Índice general

1 Apuntes del Tema 8: Descenso de Gradiente, Métodos Acelerados y Adaptativos **2**

1.1 1. Fundamentos de Optimización en Deep Learning 2

1.1.1 1.1 Optimización Matemática vs. Aprendizaje Profundo 2

1.1.2 1.2 Retos en la Superficie de Pérdida 2

1.1.3 1.3 Convexidad y Series de Taylor 2

1.2 2. Algoritmos de Descenso de Gradiente 3

1.2.1 2.1 Descenso de Gradiente Unidimensional y Multivariante 3

1.2.2 2.2 Variantes del Descenso de Gradiente según el Tamaño del Lote . . . 3

1.2.3 2.3 Método de Newton (Segundo Orden) 3

1.3 3. Optimizadores Acelerados y Adaptativos 4

1.3.1 3.1 Método del Momento (*Momentum*) 4

1.3.2 3.2 AdaGrad (*Adaptive Gradient Algorithm*) 4

1.3.3 3.3 RMSProp y AdaDelta 4

1.3.4 3.4 Adam (*Adaptive Moment Estimation*) 5

1.4 4. Polítas de Ajuste (*Learning Rate Schedulers*) y Warmup 5

1.4.1 4.1 Planificadores de Tasa de Aprendizaje (*Schedulers*) 5

1.4.2 4.2 Calentamiento (*Warmup*) 6

1.5 5. Práctica y Aplicación en Código (PyTorch) 6

1 Apuntes del Tema 8: Descenso de Gradiente, Métodos Acelerados y Adaptativos

1.1 1. Fundamentos de Optimización en Deep Learning

1.1.1 1.1 Optimización Matemática vs. Aprendizaje Profundo

- **Optimización Pura:** Su objetivo principal es minimizar una función objetivo $\mathcal{L}(w)$ sobre un conjunto de datos de entrenamiento para reducir el error de entrenamiento.
- **Deep Learning:** Su verdadero objetivo es reducir el **error de generalización** (desempeño en datos no vistos del conjunto de prueba), combinando la minimización del error con técnicas para mitigar el sobreajuste (*overfitting*).

1.1.2 1.2 Retos en la Superficie de Pérdida

1. **Mínimos Locales:** Puntos donde el gradiente es cero ($\nabla f(x) = 0$) y la matriz Hessiana es definida positiva. En redes profundas con alta dimensionalidad, la mayoría de los puntos críticos no son mínimos locales, sino puntos de silla.

1. **Puntos de Silla (*Saddle Points*):** Puntos donde el gradiente se anula, pero la matriz Hessiana posee autovalores tanto positivos como negativos (la función sube en algunas direcciones y baja en otras).

1. **Desvanecimiento del Gradiente (*Vanishing Gradients*):** Regiones extremadamente llanas de la superficie de pérdida (por ejemplo, en las zonas de saturación de $\tanh(x)$ o la sigmoide) donde $\|\nabla f(x)\| \approx 0$, ralentizando drásticamente la actualización de los parámetros.

```
Superficie de Pérdida

          /\
         /  \   <- Máximo Local
    ____/    \____
   /              \  <- Punto de Silla (Hessiana con autovalores mixtos)
  /                \____
               Mínimo Local / Global
```

1.1.3 1.3 Convexidad y Series de Taylor

- **Convexidad:** Un conjunto $\mathcal{X}$ es convexo si el segmento que une cualquier par de puntos $a, b \in \mathcal{X}$ está totalmente contenido en $\mathcal{X}$. Una función f es convexa si:

$$\lambda f(x) + (1-\lambda) f(x') \geq f(\lambda x + (1-\lambda)x'), \quad \forall \lambda \in [0,1]$$

- *Propiedad clave:* En funciones estrictamente conexas, todo mínimo local es también un **mínimo global**. Aunque las funciones de pérdida en redes profundas son no convexas, exhiben comportamiento convexo en las vecindades inmediatas de los mínimos locales.

- **Expansión en Serie de Taylor:** Aproximación polinomial local de una función diferenciable $f(x)$ alrededor de un punto x_0:

$$f(x) \approx f(x_0) + \nabla f(x_0)^\top (x - x_0) + \frac{1}{2}(x - x_0)^\top \mathbf{H}(x - x_0)$$

donde $\mathbf{H}$ es la matriz Hessiana de segundas derivadas parciales ($\mathbf{H}_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$).

1.2 2. Algoritmos de Descenso de Gradiente

1.2.1 2.1 Descenso de Gradiente Unidimensional y Multivariante

Partiendo de la expansión de Taylor de primer orden $f(x+\epsilon) \approx f(x)+\epsilon\nabla f(x)$, para garantizar la máxima disminución del valor de f, debemos movernos en la dirección opuesta al gradiente ($\epsilon = -\eta\nabla f(x)$ con $\eta > 0$):

$$w_{t+1} = w_t - \eta \nabla f(w_t)$$

- **Impacto de la Tasa de Aprendizaje (η):**
- Si η es demasiado pequeño: Convergencia extremadamente lenta.
- Si η es demasiado grande: Oscilaciones inestables e incapacidad para converger o divergencia.

1.2.2 2.2 Variantes del Descenso de Gradiente según el Tamaño del Lote

Variante	Tamaño del Lote (*Batch Size*)	Ventajas	Inconvenientes
Batch Gradient Descent	Muestras totales N	Cálculo exacto del gradiente y convergencia suave.	Inviable en memoria RAM/GPU para datasets grandes; computacionalmente lento.

|

| **Stochastic Gradient Descent (SGD)** | 1 muestra ($|B| = 1$) | Actualizaciones muy rápidas; la varianza del estimador ayuda a escapar de mínimos locales.

| Gradientes con altísima varianza, trayectoria en "zig-zag" muy ruidosa.

|

| **Mini-Batch SGD** | Lote pequeño ($|B| \in [32, 512]$) | **Estándar en la práctica:** Aprovecha la paralelizació en GPU/vectorización y suaviza el gradiente.

| Requiere afinar la tasa de aprendizaje global η.

|

1.2.3 2.3 Método de Newton (Segundo Orden)

Aprovecha la curvatura local mediante la matriz Hessiana para calcular el paso óptimo:

$$\Delta x = -\mathbf{H}^{-1}\nabla f(x)$$

- **Ventaja:** Ajusta automáticamente la amplitud del paso sin depender fuertemente de η.
- **Desventaja:** Computacionalmente prohibitivo en Deep Learning ($O(d^3)$ para invertir $\mathbf{H}$ con millones de parámetros) y sensible a direcciones donde la Hessiana es definida negativa.

1.3 3. Optimizadores Acelerados y Adaptativos

```

Evolución de Optimizadores:

SGD    Momentum (Inercia v_t)    AdaGrad (Escalado por g²)    RMSProp (Promedio Móvil
    )    Adam (Momentum + RMSProp)
```

1.3.1 3.1 Método del Momento (*Momentum*)

Introduce una variable de velocidad v_t que acumula los gradientes pasados como un promedio móvil ponderado exponencialmente (física de una bola rodando cuesta abajo):

$$v_t \leftarrow \beta v_{t-1} + g_t$$

$$w_t \leftarrow w_{t-1} - \eta v_t$$

- **Propiedades:** Amortigua las oscilaciones en direcciones de alta varianza y acelera el avance a lo largo de barrancos de la superficie de pérdida. Típicamente $\beta \approx 0.9$.

1.3.2 3.2 AdaGrad (*Adaptive Gradient Algorithm*)

Adapta la tasa de aprendizaje de cada parámetro individualmente basándose en la suma acumulada de sus gradientes al cuadrado s_t:

$$s_t \leftarrow s_{t-1} + g_t \odot g_t$$

$$w_t \leftarrow w_{t-1} - \frac{\eta}{\sqrt{s_t + \epsilon}} \odot g_t$$

- **Propiedades:** Parámetros con actualizaciones frecuentes o grandes reciben tasas de aprendizaje efectivas más pequeñas, mientras que parámetros con actualizaciones raras o pequeñas reciben pasos más grandes.
- **Inconveniente:** s_t crece de forma monótona, haciendo que la tasa de aprendizaje efectiva tienda a cero y detenga el aprendizaje prematuramente.

1.3.3 3.3 RMSProp y AdaDelta

Resuelven la acumulación desmedida de AdaGrad sustituyendo la suma por un promedio móvil con decaimiento exponencial (*leaky average*) con parámetro $\gamma \in (0, 1)$:

$$s_t \leftarrow \gamma s_{t-1} + (1-\gamma) g_t \odot g_t$$

$$w_t \leftarrow w_{t-1} - \frac{\eta}{\sqrt{s_t + \epsilon}} \odot g_t$$

1.3.4 3.4 Adam (*Adaptive Moment Estimation*)

El optimizador más popular y robusto en la práctica. Combina los beneficios del **Momento** (estimación del primer momento / media v_t) y de **RMSProp** (estimación del segundo momento / varianza s_t), añadiendo una **corrección de sesgo** para evitar la deriva hacia cero durante las primeras iteraciones:

1. **Primer Momento (Momento):**

$$v_t \leftarrow \beta_1 v_{t-1} + (1 - \beta_1) g_t$$

1. **Segundo Momento (RMSProp):**

$$s_t \leftarrow \beta_2 s_{t-1} + (1 - \beta_2)(g_t \odot g_t)$$

1. **Corrección de Sesgo (*Bias Correction*):**

$$\hat{v}_t = \frac{v_t}{1 - \beta_1^t}, \quad \hat{s}_t = \frac{s_t}{1 - \beta_2^t}$$

1. **Actualización de Parámetros:**

$$w_t \leftarrow w_{t-1} - \frac{\eta}{\sqrt{\hat{s}_t} + \epsilon} \odot \hat{v}_t$$

Hiperparámetros por defecto estándar: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

1.4 4. Polítas de Ajuste (*Learning Rate Schedulers*) y Warmup

1.4.1 4.1 Planificadores de Tasa de Aprendizaje (*Schedulers*)

Reducen la tasa de aprendizaje η a medida que avanza el entrenamiento para evitar sobrepasar el mínimo local y permitir un ajuste fino:

- **Polynomial Decay:** $\eta_t = \eta_0 \left(1 - \frac{t}{T}\right)^p$.
- **Step / Factor Decay:** $\eta_{t+1} = \eta_t \cdot \alpha$ con $\alpha \in (0, 1)$ cada k épocas.
- **Cosine Annealing:** Modula la tasa siguiendo una curva cosenoidal decreciente.
- **ReduceLROnPlateau:** Reduce η por un factor cuando una métrica de validación deja de mejorar durante varias épocas.

1.4.2 4.2 Calentamiento (*Warmup*)

Consiste en iniciar con una tasa de aprendizaje muy pequeña durante las primeras épocas e incrementarla linealmente hasta el valor η_0 antes de aplicar cualquier estrategia de decaimiento. Previene la desestabilización inicial de los pesos cuando los gradientes son excesivamente volátiles.

1.5 5. Práctica y Aplicación en Código (PyTorch)

A continuación se incluye una implementación completa en PyTorch que compara el uso de **SGD con Momentum**, **Adam** y un **Scheduler con Warmup**:

```
1
2  import torch
3
4  import torch.nn as nn
5
6  import torch.optim as optim
7
8  from torch.optim.lr_scheduler import LambdaLR, ReduceLROnPlateau
9
10
11
12 # ------------------------------------------------------------------------
13
14 # 1. Definición de una Red de Prueba
15
16 # ------------------------------------------------------------------------
17
18 class DummyNet(nn.Module):
19
20     def __init__(self, input_dim=10, num_classes=2):
21
22         super(DummyNet, self).__init__()
23
24         self.fc = nn.Sequential(
25
26             nn.Linear(input_dim, 32),
27
28             nn.ReLU(),
29
30             nn.Linear(32, num_classes)
31
32         )
33
34
35
36     def forward(self, x):
37
38         return self.fc(x)
39
40
41
42 # ------------------------------------------------------------------------
43
44 # 2. Configuración de Optimizadores en PyTorch
45
```

```
# ------------------------------------------------------------------------

def setup_optimizers(model):

    # a) SGD con Momentum y Weight Decay

    optimizer_sgd = optim.SGD(

        model.parameters(),

        lr=0.01,

        momentum=0.9,

        weight_decay=1e-4

    )



    # b) Adam (Adaptive Moment Estimation)

    optimizer_adam = optim.Adam(

        model.parameters(),

        lr=0.001,

        betas=(0.9, 0.999),

        eps=1e-8

    )



    return optimizer_sgd, optimizer_adam



# ------------------------------------------------------------------------

# 3. Implementación de Warmup + Cosine Decay Scheduler

# ------------------------------------------------------------------------

def get_warmup_scheduler(optimizer, warmup_epochs=5, max_epochs=50):

    def lr_lambda(epoch):

        if epoch < warmup_epochs:

            # Incremento lineal durante el warmup

            return float(epoch + 1) / float(warmup_epochs)

        # Decaimiento pasados los epochs de warmup

```

```
        return 0.5 * (1.0 + torch.cos(torch.tensor((epoch - warmup_epochs) / (max_epochs - warmup_epochs) * 3.14159)))


    return LambdaLR(optimizer, lr_lambda=lr_lambda)


# -----------------------------------------------------------------------

# 4. Ejemplo de Bucle de Entrenamiento

# -----------------------------------------------------------------------

def train_demo():

    model = DummyNet()

    optimizer = optim.Adam(model.parameters(), lr=1e-3)

    scheduler = get_warmup_scheduler(optimizer, warmup_epochs=5, max_epochs=20)

    criterion = nn.CrossEntropyLoss()


    # Entrada y etiquetas de prueba

    inputs = torch.randn(16, 10)

    labels = torch.randint(0, 2, (16,))


    print("--- Demostración de Entrenamiento con Scheduler ---")

    for epoch in range(1, 21):

        optimizer.zero_grad()

        outputs = model(inputs)

        loss = criterion(outputs, labels)

        loss.backward()

        optimizer.step()


        # Obtener LR actual

        current_lr = optimizer.param_groups[0]['lr']

        print(f"Época [{epoch:2d}/20] | Loss: {loss.item():.4f} | LR: {current_lr:.6f}")

```

```
160
161
162         # Paso del scheduler al final de la época
163
164         scheduler.step()
165
166
167
168 if __name__ == "__main__":
169
170     train_demo()
```

¿Hay algún punto o demostración de este bloque de optimizadores sobre el que quieras profundizar o tienes alguna duda específica?